

HTML Helpers in ASP.NET MVC

HTML Helpers are methods in ASP.NET MVC that generate HTML elements dynamically in a view. These helpers simplify the creation of form elements and other HTML controls by providing a strongly typed and server-side way to build the HTML.

Types of HTML Helpers

Type	Description	Examples
Standard Helpers	Built-in methods for creating common HTML elements like text boxes, buttons, dropdowns, etc.	Html.TextBox(), Html.ActionLink()
Strongly Typed Helpers	Generate HTML elements tied to model properties, ensuring type safety.	Html.TextBoxFor(), Html.DropDownListFor()
Custom Helpers	User-defined methods to create reusable HTML elements tailored to specific needs.	Custom HTML helper extension methods.

Description of Each Type with Code Examples

1. Standard Helpers

- **Description:** These helpers generate HTML elements but are not tied to model properties. They are less type-safe and require manual management of field names.
- **Examples:**

```
// Controller
public ActionResult Index()
{
    ViewBag.Name = "John Doe";
    return View();
}

// View
@Html.TextBox("Name", ViewBag.Name, new { @class = "form-control",
placeholder = "Enter your name" })
```

```
@Html.ActionLink("Click Here", "Index", "Home", null, new { @class = "btn
btn-primary" })
```

2. Strongly Typed Helpers

- **Description:** These helpers are tied to the model's properties, ensuring type safety. They reduce runtime errors and are recommended for strongly typed views.
- **Examples:**

```
// Model
```

```

public class UserModel
{
    public string Name { get; set; }
}

// Controller
public ActionResult Index()
{
    var model = new UserModel { Name = "John Doe" };
    return View(model);
}

// View (Strongly Typed)
@model UserModel
@Html.TextBoxFor(m => m.Name, new { @class = "form-control", placeholder
= "Enter your name" })

```

```

// Model
public class UserModel
{
    public string Country { get; set; }
    public List<SelectListItem> Countries { get; set; }
}

// Controller
public ActionResult Index()
{
    var model = new UserModel
    {
        Countries = new List<SelectListItem>
        {
            new SelectListItem { Text = "USA", Value = "USA" },
            new SelectListItem { Text = "UK", Value = "UK" },
            new SelectListItem { Text = "Canada", Value = "Canada" }
        }
    };
    return View(model);
}

// View
@model UserModel
@Html.DropDownListFor(m => m.Country, Model.Countries, "Select a Country",
new { @class = "form-select" })

```

3. Custom Helpers

- **Description:** Custom HTML helpers are user-defined extension methods to create reusable components.

- **Example:**

```
// Custom Helper Class
public static class CustomHtmlHelpers
{
    public static IHtmlString CustomButton(this HtmlHelper htmlHelper,
        string text, string type, string cssClass)
    {
        return new HtmlString($"<button type='{type}'
class='{cssClass}'>{text}</button>");
    }
}

// View
@Html.CustomButton("Submit", "submit", "btn btn-success")
```

```
// Model
public class UserModel
{
    public string Country { get; set; }
    public List<SelectListItem> Countries { get; set; }
}

// Controller
public class HomeController : Controller
{
    public ActionResult Index()
    {
        var model = new UserModel
        {
            Countries = new List<SelectListItem>
            {
                new SelectListItem { Text = "USA", Value = "USA" },
                new SelectListItem { Text = "UK", Value = "UK" },
                new SelectListItem { Text = "Canada", Value = "Canada" }
            }
        };
        return View(model);
    }
}

// View (Index.cshtml)
@model UserModel
@Html.DropDownListFor(m => m.Country, Model.Countries, "Select a Country",
    new { @class = "form-select" })
```

```
// Custom HTML Helper Class
public static class CustomHtmlHelpers
```

```

{
    public static IHtmlString CustomButton(this HtmlHelper htmlHelper,
string text, string type, string cssClass)
    {
        return new HtmlString($"<button type='{type}'
class='{cssClass}'>{text}</button>");
    }
}

// Controller
public class HomeController : Controller
{
    public ActionResult Index()
    {
        return View();
    }
}

// View (Index.cshtml)
@Html.CustomButton("Click Me", "button", "btn btn-primary")

```

```

// Custom HTML Helper Class
public static class CustomHtmlHelpers
{
    public static IHtmlString CustomLabel(this HtmlHelper htmlHelper,
string labelText, string cssClass)
    {
        return new HtmlString($"<label
class='{cssClass}'>{labelText}</label>");
    }
}

// Controller
public class HomeController : Controller
{
    public ActionResult Index()
    {
        return View();
    }
}

// View (Index.cshtml)
@Html.CustomLabel("Name:", "form-label")

```

Strongly Typed HTML Helpers in ASP.NET MVC

Strongly Typed HTML Helpers in ASP.NET MVC are designed to provide type safety and IntelliSense for creating form fields and UI elements in a view. These helpers are bound to a specific model, ensuring that field names and values correspond to properties of the model.

Benefits of Strongly Typed HTML Helpers

1. **Type Safety:** Validates bindings at compile-time, reducing runtime errors.
2. **IntelliSense Support:** Provides property suggestions while coding, improving developer productivity.
3. **Automatic Binding:** Ensures seamless two-way data binding between the view and model.

Helper	Description	Example
Html.TextBoxFor	Generates a text input field bound to a model property.	@Html.TextBoxFor(m => m.Name)
Html.TextAreaFor	Generates a multi-line text box for a model property.	@Html.TextAreaFor(m => m.Description)
Html.PasswordFor	Generates a password input field for a model property.	@Html.PasswordFor(m => m.Password)
Html.CheckBoxFor	Generates a checkbox for a boolean model property.	@Html.CheckBoxFor(m => m.IsActive)
Html.RadioButtonFor	Generates a radio button for a model property.	@Html.RadioButtonFor(m => m.Gender, "Male")
Html.DropDownListFor	Generates a dropdown list for a model property with a list of options.	@Html.DropDownListFor(m => m.Country, Model.Countries)
Html.LabelFor	Generates a label for a model property.	@Html.LabelFor(m => m.Name)
Html.HiddenFor	Generates a hidden input field for a model property.	@Html.HiddenFor(m => m.Id)
Html.EditorFor	Generates a complete input element based on the model property type.	@Html.EditorFor(m => m.Name)
Html.DisplayFor	Renders the display value of a model property as plain text.	@Html.DisplayFor(m => m.Email)

Example Code: Implementing Strongly Typed HTML Helpers

1. Create the Model

```
namespace YourNamespace.Models
{
    public class User
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
    }
}
```

```

        public string Password { get; set; }
        public bool IsActive { get; set; }
        public string Gender { get; set; }
    }
}

```

2. Create the Controller

```

using System.Web.Mvc;
using YourNamespace.Models;

namespace YourNamespace.Controllers
{
    public class UserController : Controller
    {
        public ActionResult Create()
        {
            return View(new User());
        }

        [HttpPost]
        public ActionResult Create(User user)
        {
            if (ModelState.IsValid)
            {
                // Save the user to the database (example purpose)
                return RedirectToAction("Details", new { id = user.Id });
            }
            return View(user);
        }
    }
}

```

4. Create the Strongly Typed View

```

@model YourNamespace.Models.User

<!DOCTYPE html>
<html>
<head>
    <title>Create User</title>
</head>
<body>
    <h1>Create User</h1>

    @using (Html.BeginForm())
    {
        <div>

```

```

        @Html.LabelFor(m => m.Name)
        @Html.TextBoxFor(m => m.Name)
    </div>

    <div>
        @Html.LabelFor(m => m.Email)
        @Html.TextBoxFor(m => m.Email)
    </div>

    <div>
        @Html.LabelFor(m => m.Password)
        @Html.PasswordFor(m => m.Password)
    </div>

    <div>
        @Html.LabelFor(m => m.IsActive)
        @Html.CheckBoxFor(m => m.IsActive)
    </div>

    <div>
        <label>Gender</label>
        @Html.RadioButtonFor(m => m.Gender, "Male") Male
        @Html.RadioButtonFor(m => m.Gender, "Female") Female
    </div>

    <button type="submit">Submit</button>
}
</body>
</html>

```

Implementation of Strongly Typed HTML Helpers

```

namespace YourNamespace.Models
{
    public class User
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
        public string Password { get; set; }
        public bool IsActive { get; set; }
        public string Gender { get; set; }
        public string Country { get; set; }
    }
}

```

```

using System.Web.Mvc;
using YourNamespace.Models;

```

```

namespace YourNamespace.Controllers
{
    public class UserController : Controller
    {
        public ActionResult Create()
        {
            return View(new User());
        }

        [HttpPost]
        public ActionResult Create(User user)
        {
            if (ModelState.IsValid)
            {
                // Logic to save the user (e.g., database)
                return RedirectToAction("Details", new { id = user.Id });
            }
            return View(user);
        }
    }
}

```

```

@model YourNamespace.Models.User

<!DOCTYPE html>
<html>
<head>
    <title>Create User</title>
</head>
<body>
    <h1>Create User</h1>

    @using (Html.BeginForm())
    {
        <div>
            @Html.LabelFor(m => m.Name)
            @Html.TextBoxFor(m => m.Name)
        </div>

        <div>
            @Html.LabelFor(m => m.Email)
            @Html.TextBoxFor(m => m.Email)
        </div>

        <div>
            @Html.LabelFor(m => m.Password)
            @Html.PasswordFor(m => m.Password)
        </div>
    }

```

```

        <div>
            @Html.LabelFor(m => m.IsActive)
            @Html.CheckBoxFor(m => m.IsActive)
        </div>

        <div>
            <label>Gender</label>
            @Html.RadioButtonFor(m => m.Gender, "Male") Male
            @Html.RadioButtonFor(m => m.Gender, "Female") Female
        </div>

        <div>
            @Html.LabelFor(m => m.Country)
            @Html.DropDownListFor(m => m.Country, new SelectList(new[] {
"USA", "UK", "Canada" }))
        </div>

        <button type="submit">Submit</button>
    }
</body>
</html>

```

HTML Helper Method	Description	Example Code
Html.ActionLink()	Generates a hyperlink (<a> tag) that links to a specified action method.	@Html.ActionLink("Home", "Index", "Home", null, new { @class = "btn btn-link" })
Html.BeginForm()	Creates an HTML <form> element that posts data to the server.	@using (Html.BeginForm("Submit", "Home", FormMethod.Post)) { /* form fields */ }
Html.TextBox()	Generates an HTML <input> element of type "text".	@Html.TextBox("Name", "John Doe", new { @class = "form-control" })
Html.TextBoxFor()	Generates an HTML <input> element of type "text" for a model property.	@Html.TextBoxFor(m => m.Name, new { @class = "form-control", placeholder = "Enter name" })
Html.Password()	Creates an HTML <input> element of type "password".	@Html.Password("Password", null, new { @class = "form-control" })
Html.PasswordFor()	Creates an HTML <input> element of type "password" for a model property.	@Html.PasswordFor(m => m.Password, new { @class = "form-control" })
Html.TextArea()	Generates an HTML <textarea> element for multiline input.	@Html.TextArea("Description", "Default text", new { @class = "form-control" })
Html.TextAreaFor()	Generates an HTML <textarea> element for a model property.	@Html.TextAreaFor(m => m.Description, new { @class = "form-control" })
Html.CheckBox()	Creates an HTML <input> element of type "checkbox".	@Html.CheckBox("IsActive", true)

Html.CheckBoxFor()	Creates an HTML <input> element of type "checkbox" for a model property.	@Html.CheckBoxFor(m => m.IsActive)
Html.RadioButton()	Creates an HTML <input> element of type "radio".	@Html.RadioButton("Gender", "Male", true)
Html.RadioButtonFor()	Creates an HTML <input> element of type "radio" for a model property.	@Html.RadioButtonFor(m => m.Gender, "Male")
Html.DropDownList()	Generates a <select> element with a list of options.	@Html.DropDownList("Country", new SelectList(Model.Countries), "Select a Country")
Html.DropDownListFor()	Generates a <select> element for a model property with a list of options.	@Html.DropDownListFor(m => m.Country, new SelectList(Model.Countries), "Select a Country")
Html.ListBox()	Creates an HTML <select> element that allows multiple selections.	@Html.ListBox("Options", new MultiSelectList(Model.Options))
Html.ListBoxFor()	Creates a <select> element for a model property, allowing multiple selections.	@Html.ListBoxFor(m => m.SelectedOptions, new MultiSelectList(Model.Options))
Html.Hidden()	Generates a hidden <input> element for passing data.	@Html.Hidden("Id", 123)
Html.HiddenFor()	Generates a hidden <input> element for a model property.	@Html.HiddenFor(m => m.Id)
Html.ValidationMessage()	Displays a validation message for a specific field.	@Html.ValidationMessage("Name", "Name is required.")
Html.ValidationMessageFor()	Displays a validation message for a model property.	@Html.ValidationMessageFor(m => m.Name)
Html.Partial()	Renders a partial view inside the main view.	@Html.Partial("_PartialViewName")
Html.RenderPartial()	Renders a partial view directly into the response stream.	@{ Html.RenderPartial("_PartialViewName"); }
Html.Editor()	Creates an appropriate input element based on the model's data type.	@Html.Editor("Name")
Html.EditorFor()	Creates an appropriate input element for a model property based on its data type.	@Html.EditorFor(m => m.Name)
Html.Display()	Displays the value of a model property in a read-only manner.	@Html.Display("Name")
Html.DisplayFor()	Displays the value of a model property in a strongly typed manner.	@Html.DisplayFor(m => m.Name)
Html.Label()	Generates an HTML <label> element for a field.	@Html.Label("Name", "Full Name")

Html.LabelFor()	Generates an HTML <label> element for a model property.	@Html.LabelFor(m => m.Name)
------------------------	---	-----------------------------